BÀI TẬP LÝ THUYẾT

TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Đào Duy Hưng
MSSV / Lớp	3122410158

Mục tiêu học tập

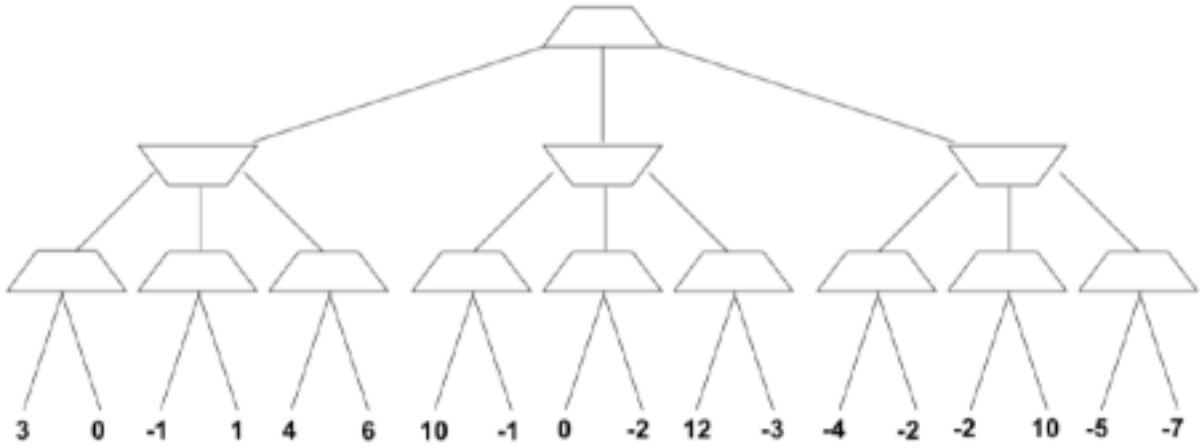
• Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa. • Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.

• Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP. • Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi. • Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Tầng MAX từ trái sang phải có các giá trị:

$$3,\ 1,\ 6,\ 10,\ 0,\ 12,\ -2,\ 10,\ -5$$

Ba nút MIN có giá trị:

$$1,\ 0,\ -5$$

Nút gốc MAX:

$$\max(1, 0, -5) = 1$$

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

MAX chọn nhánh trái vì ba nhánh có giá trị lần lượt $1, 0, -5$. Giá trị lớn nhất là 1, do đó giá trị Minimax của toàn bộ cây bằng 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Bảng tóm tắt α, β

Nút	**α khi vào**	**β khi vào**	**Kết quả**
Root MAX	$-\infty$	$+\infty$	1
MIN trái	$-\infty$	$+\infty$	1
MAX (3,0)	$-\infty$	$+\infty$	3
MAX (-1,1)	$-\infty$	3	1
MAX (4,6)	$-\infty$	1	cutoff sau 4
MIN giữa	1	$+\infty$	0
MAX (10,-1)	1	$+\infty$	10

MAX (0,-2)	1	10	0
MIN phải	1	$+\infty$	-2 / cutoff
MAX (-4,-2)	1	$+\infty$	-2

d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

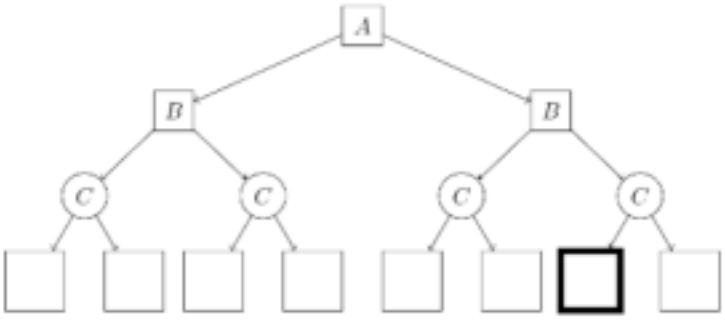
Các lá bị cắt tỉa là:

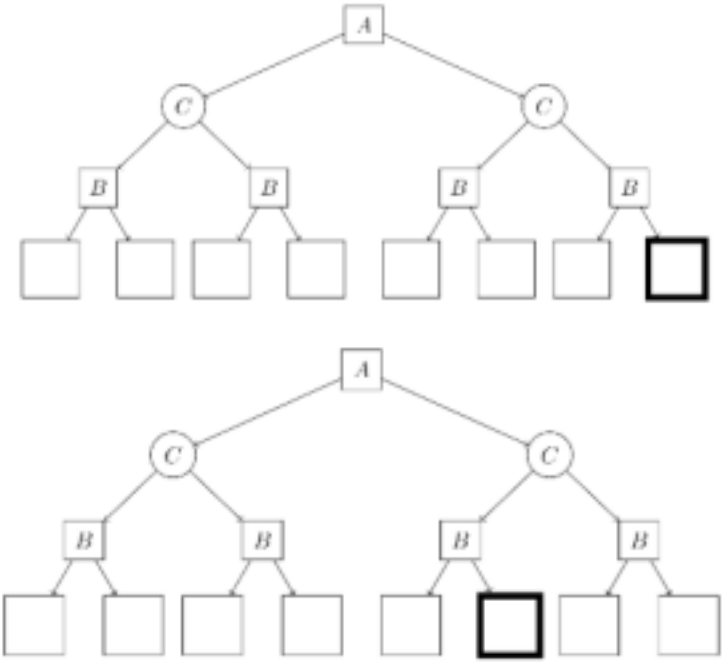
$$6,\ 12,\ -3,\ -2,\ 10,\ -5,\ -7$$

Tổng cộng có 7 lá không cần thăm. Minimax đánh giá toàn bộ 18 lá, trong khi Alpha-Beta chỉ cần đánh giá $18 - 7 = 11$ lá. Alpha-Beta vẫn cho cùng giá trị trò chơi 1 như Minimax nhưng giảm số nút phải đánh giá.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.





Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể. Ví dụ gán các lá từ trái sang phải:

$$0,\ 0,\ 0,\ 0,\ 2,\ 2,\ 5,\ 5$$

Nhánh trái cho $B_L = 0$, nên tại nút MIN A có $\beta = 0$. Ở nhánh phải, chance node đầu tiên có $C = \frac{2+2}{2} = 2$ nên $B_R \geq 2 > \beta = 0$. Vì vậy có thể cắt toàn bộ chance node còn lại của B_R, bao gồm nút viền đậm.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể. Ví dụ gán:

$$0,\ 0,\ 0,\ 0,\ 2,\ 2,\ 2,\ 5$$

Nhánh trái cho $C_L = 0$, nên tại A có $\beta = 0$. Ở nhánh phải, MAX đầu tiên bằng 2; MAX thứ hai sau khi xét lá đầu tiên cũng có cận dưới bằng 2. Vì vậy $C_R \geq \frac{2+2}{2} = 2 > 0$ nên nhánh phải không thể được MIN chọn. Lá viền đậm còn lại có thể bị cắt tỉa.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể. Khi đến nút viền đậm, mới chỉ có cận dưới cho MAX đầu tiên, trong khi

MAX thứ hai của cùng chance node vẫn hoàn toàn chưa biết. Vì các giá trị lá chỉ được yêu cầu hữu hạn nhưng không có cận dưới chung, MAX thứ hai có thể đủ âm để làm giá trị kỳ vọng của chance node nhỏ hơn giá trị hiện tại tại MIN. Vì vậy chưa thể cắt nút viền đậm một cách an toàn.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong Minimax, MIN/MAX trực tiếp chọn min hoặc max nên khi α và β chứng minh một nhánh không thể ảnh hưởng đến quyết định, có thể cắt ngay. Với chance node, giá trị được tính theo kỳ vọng $V(C) = \sum_i p_i V_i$ nên các nhánh chưa duyệt vẫn có thể làm thay đổi giá trị của nút. Vì vậy muốn cắt tỉa phải có đủ cận trên/cận dưới để chứng minh rằng các kết quả chưa biết không thể thay đổi quyết định của MIN/MAX tổ tiên.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. ☑ Đúng ☐ Sai

Tại mỗi nút MIN $\min_i V_i \leq \sum_i p_i V_i$ vì kỳ vọng luôn không nhỏ hơn giá trị nhỏ nhất. Thay các nút MIN bằng chance node do đó không làm giảm giá trị của cây, nên $V_M \leq V_E$

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít
nhất là V_E. ☑ Đúng ☐ Sai

Chính sách Minimax tối ưu bảo đảm giá trị V_M trước trường hợp MIN luôn chọn kết quả xấu nhất. Khi MIN được thay bằng chance node $E[V] \geq \min_i V_i$ nên sử dụng chính sách

Minimax vẫn cho lợi ích kỳ vọng ít nhất là V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất

là V_E. ☐ Đúng ☑ Sai

Chính sách Minimax tối ưu cho trường hợp xấu nhất, không nhất thiết tối ưu kỳ vọng. V_E là giá trị kỳ vọng tối đa khi biết phân phối xác suất, nên chính sách Minimax có thể cho $V_M \leq E[V \mid \pi_M] < V_E$ Vì vậy không đảm bảo lợi ích kỳ vọng ít nhất bằng V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi

thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi

theo giả định cần thiết.

☑ Minimax với tiện ích trạng thái cuối +1/−1

☑ Expectimax với tiện ích trạng thái cuối +1/−1

☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Có thể chọn **Minimax, Expectimax và Value Iteration** tùy giả định. Minimax phù hợp nếu xem sự bất định như một đối thủ chọn kết quả xấu nhất. Expectimax phù hợp khi biết xác suất của các kết quả và muốn tối đa hóa expected utility. Value Iteration phù hợp khi trò chơi được mô hình hóa thành MDP với xác suất chuyển trạng thái đã biết, reward 0 ở trạng thái trung gian và $+1/-1$ ở trạng thái thắng/thua.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).

Đồ thị có 6 đỉnh A, B, C, D, E, F và các cạnh:

$(A, B), (A, C), (A, D), (B, C), (B, E), (C, F)$

b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Sau khi gán A, Forward Checking chỉ xét các biến chưa gán kề trực tiếp với A. Do đó miền của B, C, D có thể bị thu hẹp.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi
Forward Checking tại B?

Khi gán B, các biến chưa gán nối trực tiếp với B là C và E. Vì vậy Forward Checking tại B có thể thay đổi miền của:

$$C,\ E$$

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Arc Consistency lan truyền thay đổi miền qua nhiều cạnh. Sau khi gán A, B, C, D có thể thay đổi trực tiếp; sự thay đổi của B, C có thể tiếp tục lan tới E, F. Vì vậy miền của:

$$B, C, D, E, F$$

đều có thể thay đổi.

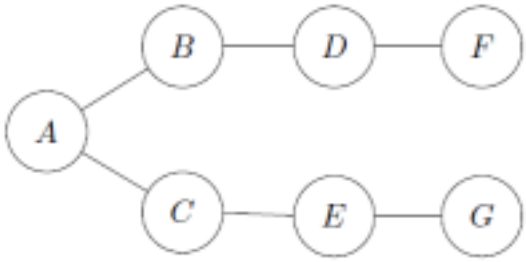
e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào
có thể thay đổi?

Sau khi gán B, Arc Consistency trước hết có thể giảm miền của C, E. Thay đổi tại C có thể tiếp tục truyền tới F. Vì vậy các miền có thể thay đổi là:

$$C, E, F$$

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi

biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến

f) Thứ tự A → B → C → D → E → F → G

Với thứ tự $A \to B \to C \to D \to E \to F \to G$, các biến được gán theo hướng từ root A xuống các nút con. Vì đồ thị là cây và đã được Arc Consistency ban đầu nên không cần backtrack 0

g) Thứ tự G → E → C → A → B → D → F

Thứ tự $G \to E \to C \to A \to B \to D \to F$ cũng là một thứ tự root-to-leaf của cây nếu lấy G làm root. Vì vậy không cần backtrack 0

h) Thứ tự G → F → E → D → C → B → A

Đây là trường hợp khác.

Thứ tự:

$$G,\ F,\ E,\ D,\ C,\ B,\ A$$

đi đồng thời từ **hai đầu của cây vào giữa**.

Các lần chạy Arc Consistency xảy ra sau biến thứ:

$$2,\ 4,\ 6$$

tức sau:

$$F,\ D,\ B$$

Sau G rồi F

G và F nằm ở hai đầu khác nhau của cây.

Sau khi gán G, chưa chạy lại AC.

Tiếp tục gán F. Giá trị của F có thể không đồng thời mở rộng được thành một nghiệm chung với giá trị đã chọn của G.

Sau F, Arc Consistency được chạy và phát hiện điều này.

Trong trường hợp xấu nhất, F có thể phải thử tới d giá trị.

Số lần quay lại $d\ -\ 1$

Sau E rồi D

Sau lần AC trước, gán:

$$E$$

rồi:

$$D$$

Tương tự, sau khi gán D, AC mới được chạy.

D có thể cần thay đổi giá trị tối đa $d\ -\ 1$ lần.

Sau C rồi B

Tiếp tục:

$$C,\ B$$

Sau B, Arc Consistency lại được chạy.

B có thể phải backtrack tối đa $d - 1$ lần.

Tổng số lần backtrack

Ba biến có khả năng phải backtrack là:

$$F,\ D,\ B$$

Mỗi biến tối đa:

$$d - 1$$

nên:

$$Backtrack_{max} = (d - 1) + (d - 1) + (d - 1)$$

$$Backtrack_{max} = 3(d - 1)$$

Các backtrack này **không lồng nhau**, vì ngay sau mỗi biến chẵn F, D, B, Arc Consistency được chạy và phát hiện vấn đề trước khi tiếp tục sâu hơn.

Kết quả

$$3(d - 1)$$

Nếu giảng viên định nghĩa "mỗi giá trị thử thất bại = một lần backtrack", kết quả tương ứng sẽ được viết:

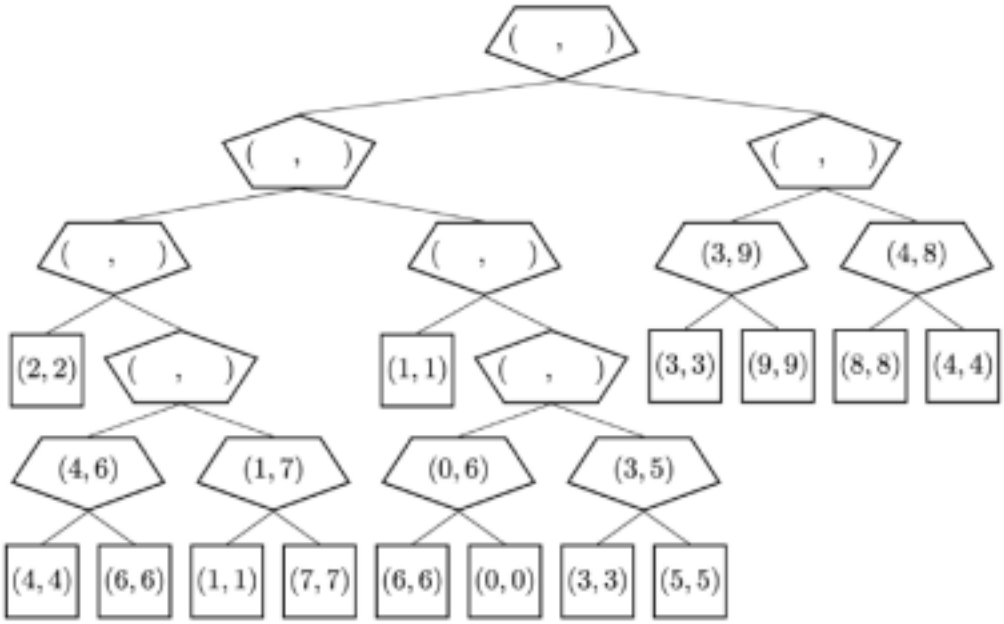
$$3d$$

nhưng với cách đếm "quay lại để thử giá trị kế tiếp", đáp án là:

$$3(d - 1)$$

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Ta tính từ dưới lên.

Hai nút Pacman ở phần dưới bên trái

Nút thứ nhất có hai con:

$$(4, 6), (1, 7)$$

Pacman so sánh thành phần u:

$$4 > 1$$

nên chọn nhánh:

$$(4, 6)$$

Do đó nút này có:

$$(4, 6)$$

Nút thứ hai có:

$$(0, 6), (3, 5)$$

Vì:

$$3 > 0$$

Pacman chọn:

$$(3, 5)$$

Nút Casper bên trái

Hai con là:

$$(2, 2), (4, 6)$$

Theo niềm tin của Pacman:

$$u = \min(2, 4) = 2$$

Theo hành vi thực tế của Casper:

$$v = \max(2, 6) = 6$$

Do đó:

$$(2, 6)$$

Nút Casper kế bên

Hai con:

$$(1, 1), (3, 5)$$

Ta có:

$$u = \min(1, 3) = 1$$

$$v = \max(1, 5) = 5$$

Do đó:

$$(1, 5)$$

Nút Pacman phía trên bên trái

Hai con:

$$(2, 6), (1, 5)$$

Pacman chọn theo u:

$$2 > 1$$

nên chọn nhánh đầu tiên:

$$(2, 6)$$

Nút Pacman phía trên bên phải

Hai con đã được cho:

$$(3, 9), (4, 8)$$

Pacman chỉ nhìn vào u:

$$4 > 3$$

nên chọn:

$$(4, 8)$$

Đây là điểm rất quan trọng: mặc dù nhánh $(3, 9)$ thực tế đem lại tiện ích 9, Pacman tin nó chỉ đáng giá 3, nên lại chọn nhánh $(4, 8)$.

Nút gốc Casper

Hai con:

$$(2, 6), (4, 8)$$

Theo niềm tin của Pacman, Casper chọn MIN:

$$u = \min(2, 4) = 2$$

Nhưng Casper thực tế muốn tăng lợi ích cho Pacman:

$$v = \max(6, 8) = 8$$

Do đó nút gốc là:

$$(2, 8)$$

Các cặp cần điền vào Hình 4

Vị trí	Giá trị
Pacman dưới bên trái	$(4, 6)$
Pacman dưới ở giữa	$(3, 5)$
Casper phía trên nhánh trái	$(2, 6)$
Casper phía trên nhánh giữa	$(1, 5)$

Pacman tầng trên bên trái	$(2, 6)$
Pacman tầng trên bên phải	$(4, 8)$
Nút gốc Casper	$(2, 8)$

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Khi duyệt từ trái sang phải, tại nút Pacman có lựa chọn hiện tại $u = 4$, nhánh Casper kế tiếp sau khi thấy lá 1 có $u \leq 1 < 4$, nên có thể bỏ lá 7. Tương tự, tại Pacman phía trên đã có $u = 2$; nhánh Casper còn lại sau khi thấy lá 1 có $u \leq 1 < 2$, nên toàn bộ nhánh chứa các lá $6, 0, 3, 5$ không cần duyệt. Các lá có thể bỏ qua là:

$$7,\ 6,\ 0,\ 3,\ 5$$

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Nếu Pacman biết Casper thân thiện thì Pacman không còn đánh giá Casper như MIN. Cả Pacman và Casper đều chọn hành động làm cực đại tiện ích. Khi đó nhánh có tiện ích 9 được chọn thay cho nhánh có tiện ích 8. Vì vậy giá trị thực của toàn bộ cây tăng:

$$8 \rightarrow 9$$

và nếu tiếp tục dùng ký hiệu cặp thì giá trị gốc là:

$$(9, 9)$$

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu

Breadth-First Search (BFS)	Branching factor hữu hạn và goal ở độ sâu hữu hạn; Graph Search trên không gian hữu hạn với duplicate detection là complete.	Optimal nếu mọi bước có cùng chi phí dương.
Depth-First Search (DFS)	Tree Search: chỉ bảo đảm khi độ sâu hữu hạn. Graph Search: complete trên không gian trạng thái hữu hạn với explored set.	Không được đảm bảo optimal nói chung.
Iterative Deepening Search (IDS)	Branching factor hữu hạn và goal ở độ sâu hữu hạn.	Optimal nếu mọi bước có cùng chi phí dương.
A* Search	Branching hữu hạn, nghiệm hữu hạn và $c(n, a, n') \geq \varepsilon > 0$.	Tree Search: h admissible. Graph Search: h consistent nếu không reopen; admissible có thể đủ nếu cho phép reopen.

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

Chi phí bước ảnh hưởng đến optimality vì BFS và IDS tối ưu theo độ sâu chứ không theo path cost; chúng chỉ đảm bảo tối ưu khi các bước có cùng chi phí. Với A*, điều kiện $c \geq \varepsilon > 0$ giúp bảo đảm thuật toán không mở rộng vô hạn nhiều node có chi phí bị chặn. Đặc tính heuristic cũng quyết định optimality của A*: Tree Search cần heuristic admissible $h(n) \leq h^*(n)$, còn Graph Search chuẩn không reopen node cần heuristic consistent $h(n) \leq c(n, n') + h(n')$. Consistency là điều kiện mạnh hơn admissibility.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Vì $X \rightarrow Y$ nằm trên đường tối ưu cũ nên chi phí đường cũ tăng từ C^* thành $C^* + n$. Đường cũ có thể không còn tối ưu:

$$C_{new} = \min(C^* + n, C_{alt})$$

Cập nhật $g(Y)$ và các hậu duệ chịu ảnh hưởng, reopen các node cần thiết và tiếp tục A* từ frontier cũ thay vì chạy lại toàn bộ.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí tối ưu chắc chắn giảm đúng n:

$$C_{new} = C^* - n$$

vì đường tối ưu cũ sử dụng cạnh này, trong khi không có đường nào khác có thể trở nên tốt hơn tương đối. Chỉ cần cập nhật cost của cạnh và các g-value trên phần đường bị ảnh hưởng.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường tối ưu cũ không dùng $X \rightarrow Y$, nên nó không bị ảnh hưởng. Các đường sử dụng cạnh này chỉ trở nên đắt hơn. Vì vậy:

$$C_{new} = C^*$$

Không cần chạy lại A* để tìm nghiệm hiện tại; chỉ cập nhật dữ liệu tìm kiếm nếu muốn tái sử dụng sau này.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Chi phí tối ưu **có thể giảm hoặc giữ nguyên**:

$$C^* - n \leq C_{new} \leq C^*$$

Relax lại cạnh:

$$g'(Y) = g(X) + (c - n)$$

Nếu $g'(Y) < g(Y)$, cập nhật $g(Y)$, reopen Y và tiếp tục A* từ vùng bị ảnh hưởng. Nếu không cải thiện, giữ đường tối ưu cũ.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Vì cạnh không nằm trên đường tối ưu và chi phí chỉ tăng nên không thể tạo ra đường tốt hơn. Do đó,

$$C_{new} = C^*$$

Không cần tìm kiếm lại; chỉ cập nhật chi phí cạnh trong mô hình.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Chi phí tối ưu có thể giảm:

$$C^* - n \leq C_{new} \leq C^*$$

Vì cạnh chưa được khám phá, chưa chắc có đủ thông tin để relax trực tiếp. Cần đưa vùng chứa $X \rightarrow Y$ vào frontier và tiếp tục tìm kiếm; có thể tái sử dụng kết quả A* cũ nhưng trong trường hợp xấu nhất phải khám phá thêm một phần lớn không gian tìm kiếm.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ: • A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Các lớp tác tử: B và C. Vì môi trường quan sát đầy đủ và xác định nên percept hiện tại đủ để xác định trạng thái và hành động tối ưu; do đó B có thể hành động hợp lý. C cũng có thể vì có thể sử dụng hoặc bỏ qua lịch sử percept. A không dùng percept nên không đảm bảo hành động đúng khi các trạng thái yêu cầu hành động khác nhau.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử: C. Trong môi trường quan sát một phần, percept hiện tại có thể không đủ xác định trạng thái. Tác tử C sử dụng chuỗi percept để suy luận trạng thái/belief state và chọn hành động hợp lý. B chỉ dùng percept hiện tại nên có thể không phân biệt được các trạng thái khác nhau; A không dùng percept.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Lớp tác tử: C. Khi môi trường vừa quan sát một phần vừa ngẫu nhiên, tác tử cần dùng lịch sử percept để duy trì belief state và lựa chọn hành động tối đa hóa lợi ích kỳ vọng:

$$a^* = arg\,max_a E[U|b_t, a]$$

B chỉ dùng percept hiện tại nên nói chung không đủ; A không dùng percept nên càng không đủ.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Ví dụ một môi trường trong đó mọi hành động đều cho cùng một reward và dẫn đến cùng một kết quả. Khi:

$$U(a_1) = U(a_2) = \cdots = U(a_k)$$

thì mọi hành động đều tối ưu, nên bất kỳ tác tử nào, kể cả tác tử lựa chọn ngẫu nhiên, cũng hành động hợp lý.

— Hết —